Nonstationarity-Aware Mamba for Time Series Forecasting

Draft Manuscript

May 2, 2026

Abstract

Nonstationary time series forecasting is difficult because the data-generating process can change across time. Changes in level, volatility, trend, and temporal regime can make a model trained on past windows unreliable for future horizons. This manuscript studies nonstationarity-aware forecasting with Mamba-based sequence models, using daily GLD close-price prediction as the initial testbed. The proposed model families extend an S_Mamba backbone with adaptive normalization, series decomposition, distribution-shift-aware fusion, and multi-scale Mamba branches. Before evaluating the proposed architecture, we define a controlled baseline protocol across 12 horizons from 1 to 360 days and lock a shared configuration through S_Mamba calibration. The completed GLD baseline results show that S_Mamba is the strongest model from 1 to 90 days on RMSE, while iTransformer is strongest from 120 to 360 days. The first NS-Mamba diagnostic ablation shows that the decomposition-only variant improves long-horizon RMSE over both S_Mamba and iTransformer. After Mamba-specific hyperparameter optimization, MS-Mamba-MD becomes the strongest Mamba-family model and outperforms optimized NLinear and TimesNet on the completed long-horizon diagnostic average. The completed source-optimized imported-model sweep further shows that PatchTST is the strongest imported baseline overall when its own hyperparameters are tuned rather than forced into the shared setting. PatchTST remains the strongest overall long-horizon model in the current evidence, so the manuscript treats MS-Mamba-MD as the promoted Mamba candidate rather than as a final state-of-the-art claim.

1 Introduction

Time series forecasting models are often evaluated under the assumption that past and future windows are drawn from similar distributions. This assumption is weak for nonstationary real-world data. Gold-related exchange-traded products such as GLD can experience abrupt level shifts, volatility changes, macroeconomic regime changes, and long periods of trend persistence or reversal. These effects are not just noise; they change the forecasting problem itself.

Recent deep forecasting models use attention, decomposition, and state-space mechanisms to model long temporal context. Mamba is especially attractive because it provides linear-time sequence modeling through selective state spaces [1]. In a forecasting setting, however, using Mamba alone does not guarantee robustness to nonstationarity. The model may still entangle level, scale, trend, and residual dynamics in one hidden representation. This motivates an architecture that keeps the efficient Mamba backbone while explicitly exposing nonstationary structure to the network.

This work begins with a strict experimental principle. Before introducing NS-Mamba, all suitable baselines are evaluated under a shared configuration selected by S_Mamba calibration. This prevents model-specific tuning from becoming a hidden source of performance differences. The

same horizon suite and output scale are then used for clean baselines, Mamba variants, and the proposed NS-Mamba models.

The intended contributions are:

- a controlled GLD forecasting protocol covering horizons from 1 to 360 days;
- a clean comparison between Transformer-style, decomposition-based, inverted, and Mamba-based forecasting models;
- a nonstationarity-aware Mamba architecture family combining adaptive normalization, decomposition, and shift-aware fusion;
- a diagnostic ablation showing that decomposition is the strongest first modification on GLD long-horizon forecasting.

2 Related Work

2.1 Long-Horizon Time Series Forecasting

Long-horizon time series forecasting aims to predict many future steps from a historical lookback window. It is important in domains where decisions must be made before the target is observed, including finance, energy, traffic, weather, and operations planning. Compared with short-term forecasting, long-horizon forecasting is more sensitive to error accumulation, distribution shift, trend changes, and the difficulty of preserving useful long-range temporal information. In financial forecasting, these challenges are amplified because the target can move under changing macroeconomic conditions, market expectations, and volatility regimes. The GLD task studied in this manuscript follows this setting by evaluating horizons from 1 day to 360 days.

2.2 Transformer-Based Time Series Forecasting

Transformer-based forecasting models introduced attention mechanisms for modeling long temporal context. Informer improves efficiency by reducing the cost of attention for long sequences [10]. Reformer reduces attention complexity through locality-sensitive hashing and reversible layers [12]. Autoformer introduces decomposition into the forecasting architecture, separating trend and seasonal components to improve long-term prediction [11]. These models provide important baselines because they represent different design choices for efficient long-context modeling, decomposition, and temporal dependency extraction.

2.3 Inverted Forecasting Architectures

iTransformer argues that treating variables as tokens and encoding their temporal histories can be effective for multivariate forecasting [9]. The S_Mamba baseline in this repository follows a related inverted-token view by embedding each variable's lookback history before applying Mamba encoder layers. For the initial GLD setting, the task is single-target forecasting, but the same design remains relevant because it controls how temporal history is mapped into hidden features.

2.4 Mamba-Based Time Series Forecasting

Several recent studies have adapted Mamba to time series forecasting. S_Mamba studies whether Mamba is effective for time series forecasting and uses a bidirectional Mamba layer to model inter-variate correlations with a feed-forward layer for temporal dependencies [2]. Bi-Mamba+ extends

this direction with forward and backward Mamba processing, a forget gate, and a relation-aware choice between channel-independent and channel-mixing tokenization [3]. MambaTS analyzes limitations of directly applying Mamba to long-term forecasting and proposes temporal Mamba blocks, variable scanning strategies, and variable-aware scan ordering [4]. TimeMachine uses multiple Mamba pathways to handle channel-mixing and channel-independence settings while preserving linear scalability for long-term forecasting [5]. TSMamba moves from task-specific forecasting toward a Mamba-based foundation model, using forward and backward Mamba encoders and transfer learning for time series prediction [6]. Together, these examples show that Mamba has become an active backbone for forecasting, but they do not directly resolve the specific problem studied here: explicit nonstationarity handling through adaptive normalization, decomposition, and shift-aware fusion.

3 Problem Formulation and Method

Let

$$\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L\times C} \tag{1}$$

denote an input window of length L with C variables. The goal is to predict

$$\mathbf{Y}_{t+1:t+H} \in \mathbb{R}^{H\times C_o}, \tag{2}$$

where H is the forecast horizon and C_o is the output dimension. For the GLD study, $C = C_o = 1$ and the target is the daily `close` column.

The evaluated horizon set is:

$$H \in \{1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360\}. \tag{3}$$

These horizons cover short-term, middle-term, and long-term forecasting behavior.

Nonstationarity means that the joint distribution of input and output windows changes with time. Let $\mathcal{D}_t$ denote the local joint distribution around time t. The nonstationary setting can be written as:

$$\mathcal{D}_t(\mathbf{X}_{t-L+1:t}, \mathbf{Y}_{t+1:t+H}) \neq \mathcal{D}_{t+\tau}(\mathbf{X}_{t+\tau-L+1:t+\tau}, \mathbf{Y}_{t+\tau+1:t+\tau+H}). \tag{4}$$

This mismatch means that minimizing empirical training risk does not necessarily minimize future test risk. In GLD forecasting, common sources of mismatch include changing price level, volatility expansion or contraction, trend reversal, and regime-specific temporal dependence.

The proposed approach is based on three assumptions. First, local level and scale variation should be normalized before sequence modeling. Second, trend-like and residual-like movements should not always be forced into the same representation. Third, the model should receive explicit shift statistics so that it can change its fusion behavior under different regimes.

3.1 State Space Models and Mamba

State space models provide an alternative to attention for sequence modeling. The continuous-time form of a linear state space model is:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \qquad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t), \tag{5}$$

where $\mathbf{h}(t)$ is the latent state, $\mathbf{x}(t)$ is the input, and $\mathbf{y}(t)$ is the output. After discretization with step size Δ, the recurrence can be written as:

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t, \tag{6}$$

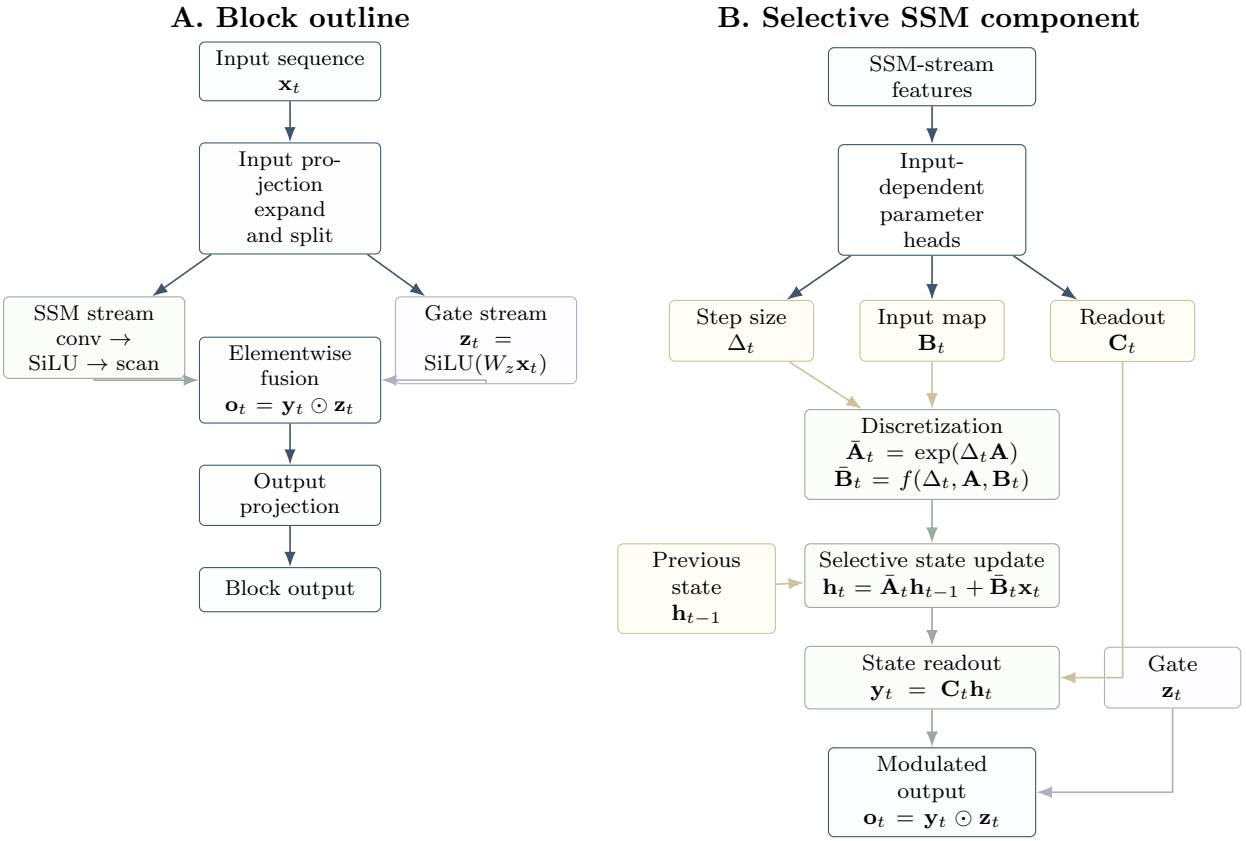


Figure 1: Mamba block architecture drawn with native TikZ so mathematical notation is rendered by LaTeX. Panel A shows the high-level block outline: input projection, SSM stream, gating stream, fusion, and output projection. Panel B expands the key selective state-space component, where input-dependent parameters generate the discretized update, readout, and gated output.

with

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \qquad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}\left(\exp(\Delta\mathbf{A}) - \mathbf{I}\right)\Delta\mathbf{B}. \tag{7}$$

Classical structured state space models use fixed transition parameters after training. Mamba introduces selective state spaces, where Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ depend on the current input token [1]. The input-dependent recurrence is:

$$\mathbf{h}_t = \bar{\mathbf{A}}_t\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}_t\mathbf{h}_t, \tag{8}$$

where $\bar{\mathbf{A}}_t$ and $\bar{\mathbf{B}}_t$ are computed from the input-dependent step size and input projection. This selectivity is useful for forecasting because the model can adapt what information is retained across time. Figure 1 summarizes the Mamba block used as the conceptual backbone in this manuscript.

3.2 Nonstationarity and Normalization

Distribution shift is a central issue in time series forecasting. RevIN uses reversible instance normalization to reduce distribution shift between training and testing windows while restoring the original scale for prediction [7]. Non-stationary Transformers introduce mechanisms to handle stationarity and nonstationarity within Transformer forecasting [8]. The central question in this manuscript is whether a Mamba backbone can benefit from the same general idea when nonstationary structure is made explicit before sequence modeling and during feature fusion.

NS-Mamba is designed as a modular extension of S_Mamba. The base model provides the sequence backbone, while the new modules expose nonstationary structure.

Given an input window $\mathbf{X}$, the model computes:

$$\hat{\mathbf{Y}} = \mathrm{DeNorm}\left(F_\theta\left(\mathrm{Decomp}\left(\mathrm{AdaptiveNorm}(\mathbf{X})\right), \phi(\mathbf{X})\right)\right), \tag{9}$$

where F_θ is the Mamba forecasting network and $\phi(\mathbf{X})$ is a vector of distribution-shift statistics.

3.3 Adaptive Normalization

The first module normalizes the input window and learns adaptive affine parameters from window statistics. The normalized input is:

$$\tilde{\mathbf{X}} = \gamma(\phi(\mathbf{X})) \odot \frac{\mathbf{X} - \mu(\mathbf{X})}{\sigma(\mathbf{X}) + \epsilon} + \beta(\phi(\mathbf{X})). \tag{10}$$

Here $\mu(\mathbf{X})$ and $\sigma(\mathbf{X})$ are local window statistics, while $\gamma(\cdot)$ and $\beta(\cdot)$ are learned functions. This module generalizes fixed reversible normalization by allowing the normalized representation to depend on distribution-shift features.

3.4 Series Decomposition

The second module decomposes the normalized window into trend, seasonal, and residual components:

$$\tilde{\mathbf{X}} = \tilde{\mathbf{X}}^{\mathrm{trend}} + \tilde{\mathbf{X}}^{\mathrm{seasonal}} + \tilde{\mathbf{X}}^{\mathrm{residual}}. \tag{11}$$

The initial implementation uses moving-average decomposition. The purpose is not to assert a perfect additive data-generating process, but to provide an architectural bias that separates slow and fast temporal variation.

3.5 Shift-Aware Fusion

The third module uses the shift statistics $\phi(\mathbf{X})$ to control the fusion of hidden representations. For branch outputs $\mathbf{z}_k$, the fusion weights are:

$$\boldsymbol{\alpha} = \mathrm{softmax}\left(\mathbf{W}_\alpha \phi(\mathbf{X}) + \mathbf{b}_\alpha\right), \tag{12}$$

and the fused representation is:

$$\mathbf{z} = \sum_k \alpha_k \mathbf{z}_k. \tag{13}$$

This design allows the model to emphasize trend, seasonal, or residual branches differently when the input window exhibits different shift patterns.

4 Neural Network Architecture

4.1 Input and Tensor Shapes

The implementation uses tensors of shape $\mathbb{R}^{B \times L \times C}$, where B is batch size. For GLD, $C = 1$. After adaptive normalization and optional decomposition, each component is passed to an inverted embedding layer. The embedding maps temporal history into hidden dimension d_{model}. In the locked GLD setting, $d_{\mathrm{model}} = 64$ and the feed-forward dimension is 128.

4.2 S_Mamba Encoder

The S_Mamba backbone applies bidirectional Mamba encoder layers to the embedded representation. A simplified state-space update is:

$$\mathbf{h}_t = \mathbf{A}(\mathbf{x}_t)\mathbf{h}_{t-1} + \mathbf{B}(\mathbf{x}_t)\mathbf{x}_t, \qquad \mathbf{o}_t = \mathbf{C}(\mathbf{x}_t)\mathbf{h}_t. \tag{14}$$

The input-dependent matrices give Mamba a selective memory mechanism. NS-Mamba preserves this mechanism and adds explicit nonstationary conditioning before and around the encoder.

4.3 Variant Definitions

This repository contains several Mamba-family variants. The names encode which nonstationary modules are active. **A** denotes adaptive normalization, **D** denotes trend–seasonal–residual decomposition, and **G** denotes shift-aware gated fusion. **M** denotes multi-scale temporal branches, **S** denotes an NLinear-style residual skip, and **P** denotes patch-temporal Mamba branches. All variants produce forecasts with shape $\mathbb{R}^{B\times H\times C}$ and use reversible output scaling when normalization is enabled.

Let

$$\mathcal{N}(\mathbf{X}) = \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma} + \epsilon}, \qquad \mathcal{N}^{-1}(\mathbf{Z}) = \mathbf{Z} \odot (\boldsymbol{\sigma} + \epsilon) + \boldsymbol{\mu}, \tag{15}$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are computed from the input window. Let $\mathcal{F}_\theta(\cdot)$ denote one inverted Mamba forecasting branch, including inverted embedding, bidirectional Mamba encoding, and horizon projection. The S_Mamba baseline is therefore:

$$\hat{\mathbf{Y}}_{\mathrm{S}} = \mathcal{N}^{-1}\left(\mathcal{F}_\theta(\mathcal{N}(\mathbf{X}))\right). \tag{16}$$

Adaptive-normalization variants. NS-Mamba-A replaces fixed normalization with adaptive normalization:

$$\mathcal{N}_A(\mathbf{X}) = \boldsymbol{\gamma}(\phi(\mathbf{X})) \odot \mathcal{N}(\mathbf{X}) + \boldsymbol{\beta}(\phi(\mathbf{X})), \tag{17}$$

where $\phi(\mathbf{X})$ contains window shift statistics such as mean shift, variance shift, slope, range, and recent volatility. The forecast is:

$$\hat{\mathbf{Y}}_{\mathrm{A}} = \mathcal{N}_A^{-1}\left(\mathcal{F}_\theta(\mathcal{N}_A(\mathbf{X}))\right). \tag{18}$$

This variant tests whether a learned normalization transform alone is enough to improve nonstationary forecasting.

Decomposition variants. NS-Mamba-D first applies standard reversible normalization and then decomposes the normalized signal:

$$\mathbf{Z} = \mathcal{N}(\mathbf{X}), \qquad (\mathbf{Z}^{\mathrm{tr}}, \mathbf{Z}^{\mathrm{se}}, \mathbf{Z}^{\mathrm{re}}) = \mathcal{D}_m(\mathbf{Z}), \tag{19}$$

where $\mathcal{D}_m$ is a moving-average decomposition with window size m. The three components represent trend, seasonal or medium-frequency variation, and residual variation. The decomposition forecast is:

$$\hat{\mathbf{Y}}_{\mathrm{D}} = \mathcal{N}^{-1}\left(\sum_{q\in\{\mathrm{tr},\mathrm{se},\mathrm{re}\}} \mathcal{F}_{\theta_q}(\mathbf{Z}^q)\right). \tag{20}$$

NS-Mamba-AD uses the same branch structure but replaces $\mathcal{N}$ with $\mathcal{N}_A$.

Shift-gated variants. NS-Mamba-ADG adds a shift-aware fusion gate to the adaptive decomposition branches. For component forecasts $\mathbf{U}_q = \mathcal{F}_{\theta_q}(\mathbf{Z}^q)$, the gate is:

$$\boldsymbol{\alpha} = \operatorname{softmax}\left(g_\psi(\phi(\mathbf{X}))\right), \qquad \sum_q \alpha_q = 1, \tag{21}$$

and the forecast is:

$$\hat{\mathbf{Y}}_{\mathrm{ADG}} = \mathcal{N}_A^{-1}\left(\sum_q \alpha_q \odot \mathbf{U}_q\right). \tag{22}$$

This variant tests whether distribution-shift statistics can improve how the model combines trend, seasonal, and residual information.

Multi-scale variants. The MS-Mamba family adds temporal scale branches. Let $\mathcal{P}_s(\cdot)$ be average-pooling downsampling by scale s, and let $\mathcal{S}$ be a set of scales such as $\{1, 2, 4\}$. For a decomposition-enabled multi-scale model, each scale produces:

$$\mathbf{U}_s = \sum_{q \in \{\mathrm{tr,se,re}\}} \mathcal{F}_{\theta_{s,q}}\left(\mathcal{P}_s(\mathbf{Z}^q)\right). \tag{23}$$

MS-Mamba-MD uses mean scale fusion:

$$\hat{\mathbf{Y}}_{\mathrm{MD}} = \mathcal{N}^{-1}\left(\frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \mathbf{U}_s\right). \tag{24}$$

MS-Mamba-MDG replaces the mean by shift-aware scale weights:

$$\boldsymbol{\pi} = \operatorname{softmax}(g_\omega(\phi(\mathbf{X}))), \qquad \hat{\mathbf{Y}}_{\mathrm{MDG}} = \mathcal{N}^{-1}\left(\sum_{s\in\mathcal{S}} \pi_s \odot \mathbf{U}_s\right). \tag{25}$$

MS-Mamba-M and MS-Mamba-MG are the corresponding multi-scale variants without the decomposition components.

Linear-skip and patch variants. MS-Mamba-MDS adds a last-value residual linear skip:

$$\mathbf{U}_{\mathrm{skip}} = \mathbf{x}_L + \mathbf{W}_{\mathrm{skip}}(\mathbf{Z} - \mathbf{x}_L), \qquad \hat{\mathbf{Y}}_{\mathrm{MDS}} = \mathcal{N}^{-1}\left(\lambda \mathbf{U}_{\mathrm{MD}} + (1-\lambda)\mathbf{U}_{\mathrm{skip}}\right), \tag{26}$$

where $\lambda = \sigma(\rho)$ is learned. PMS-Mamba replaces inverted variable-token branches with patch-temporal Mamba branches. For patch length p and stride r, a scale branch forms patch tokens:

$$\mathbf{e}_j^{(s)} = \mathbf{W}_p \operatorname{vec}\left(\mathcal{P}_s(\mathbf{Z})_{j:j+p-1}\right), \tag{27}$$

then applies Mamba over the patch index j. PMS-Mamba-PD is therefore the patch-temporal counterpart of MS-Mamba-MD.

4.4 Forecast Head and Loss

The forecast head projects the final hidden representation to H future values. The training objective is mean squared error over the forecast horizon:

$$\mathcal{L} = \frac{1}{BHC_o}\sum_{b=1}^{B}\sum_{h=1}^{H}\sum_{c=1}^{C_o}\left(\hat{Y}_{b,h,c} - Y_{b,h,c}\right)^2. \tag{28}$$

In the current codebase, inverse transformation is enabled by default, so saved predictions, targets, and metrics are reported on the original GLD price scale unless a run uses `--no-inverse`.

Table 1: Meaning of the Mamba-family variant names used in this manuscript.

Variant	Active modules	Fusion	Purpose
NS-Mamba-A	Adaptive norm	Single path	Test learned normalization
NS-Mamba-D	Decomposition	Sum	Test component separation
NS-Mamba-AD	Adaptive norm + decomposition	Sum	Combine A and D
NS-Mamba-ADG	Adaptive norm + decomposition	Shift gate	Test regime-aware component fusion
MS-Mamba-M	Multi-scale	Mean	Test temporal scales alone
MS-Mamba-MG	Multi-scale	Shift gate	Test gated scale fusion
MS-Mamba-MD	Multi-scale + decomposition	Mean	Promoted Mamba-HPO model
MS-Mamba-MDG	Multi-scale + decomposition	Shift gate	Test gated multi-scale decomposition
MS-Mamba-MDS	Multi-scale + decomposition + skip	Mean + skip	Test NLinear-style residual path
PMS-Mamba-PD	Patch Mamba + decomposition	Mean	Test patch-temporal Mamba

5 Experimental Setup

5.1 Dataset

The initial dataset is GLD daily data stored at `data/GLD/GLD.csv`. The target column is `close`. The first experimental phase uses single-target forecasting. Future extensions may add more financial or macroeconomic variables, but those are outside the current baseline protocol.

5.2 Data Splits

The repository uses the standard custom-data split from the forecasting codebase. The validation and test windows use historical lookback context from the previous split to construct the input sequence. This is a common forecasting implementation detail, but it should be reported clearly because the input context for validation and test includes observations before the split boundary.

5.3 Baseline Models

The core baseline set is:

- S_Mamba;
- Transformer;
- Informer;
- Reformer;
- Autoformer;
- iTransformer.

Optional Mamba-augmented variants will be reported separately if they are included. The clean baselines should not be tuned independently after the shared setting is locked.

Table 2: Best and second-best GLD baseline models by horizon, measured with RMSE.

Horizon	Best model	RMSE	Runner-up	RMSE
1	S_Mamba	4.45	iTransformer	4.59
3	S_Mamba	5.30	iTransformer	5.45
5	S_Mamba	5.76	iTransformer	6.12
7	S_Mamba	6.47	iTransformer	6.85
14	S_Mamba	8.17	iTransformer	8.55
30	S_Mamba	11.12	iTransformer	11.45
60	S_Mamba	14.80	iTransformer	14.89
90	S_Mamba	16.68	iTransformer	16.89
120	iTransformer	19.89	S_Mamba	20.31
180	iTransformer	26.71	S_Mamba	26.91
270	iTransformer	32.95	S_Mamba	33.00
360	iTransformer	39.45	S_Mamba	39.48

5.4 Shared Configuration

S_Mamba calibration selected Setting A as the locked baseline configuration:

Parameter	Value
Lookback length	360
Label length	180
Prediction length	horizon-specific
Model dimension	64
Feed-forward dimension	128
Encoder layers	2
Batch size	16
Target	`close`
Output scale	original price scale

All suitable models will use this configuration. If a model cannot run under this setting, the failure will be recorded rather than changing only that model's hyperparameters.

5.5 Metrics and Logging

The primary manuscript metrics are RMSE and MAE. MSE, MAPE, and MSPE are also saved when available. Training, validation, and test metrics are logged to TensorBoard. Final arrays are saved as `pred.npy`, `true.npy`, and `metrics.npy`, with run metadata stored in `meta.json`. The completed GLD baseline sweep contains 72 finished runs, corresponding to 6 models across 12 horizons.

6 Results

The GLD baseline evaluation is complete under the locked shared setting. All 72 runs finished successfully, corresponding to 6 core models evaluated across 12 horizons. The machine-readable result table for this submission package is stored in `data/gld_baselines.csv`.

The main pattern is a two-model regime. S_Mamba is the strongest RMSE baseline for 8 horizons, covering every horizon from 1 to 90 days, while iTransformer is strongest for the remaining

Table 3: Average GLD baseline RMSE grouped by forecast range. Short = 1–7 days, middle = 14–120 days, long = 180–360 days.

Model	Short Avg. RMSE	Middle Avg. RMSE	Long Avg. RMSE	Overall Avg. RMSE
S_Mamba	5.49	14.21	33.13	16.04
iTransformer	5.75	14.34	33.04	16.15
Autoformer	36.06	37.70	43.93	38.71
Transformer	69.11	66.78	74.80	69.56
Informer	79.27	75.60	72.96	76.17
Reformer	68.27	77.47	84.87	76.25

Table 4: Diagnostic GLD RMSE results for NS-Mamba variants. The long average is computed over 120, 180, and 360 days.

Model	Adaptive Norm	Decomp.	Gate	30	120	180	360	Long Avg.
S_Mamba	No	No	No	11.12	20.31	26.91	39.48	28.90
iTransformer	Yes	No	No	11.45	19.89	26.71	39.45	28.69
NS-Mamba-A	Yes	No	No	11.16	21.16	28.40	43.93	31.16
NS-Mamba-D	No	Yes	No	**9.66**	**18.25**	**24.22**	**37.74**	**26.74**
NS-Mamba-AD	Yes	Yes	No	10.46	20.50	27.49	42.39	30.13
NS-Mamba-ADG	Yes	Yes	Yes	10.64	20.19	27.60	45.50	31.10

4 horizons from 120 to 360 days. The handover is narrow rather than abrupt: at 120 days the RMSE gap is about 2.05% in favor of iTransformer, while at 360 days the gap is only about 0.06%. This means the long-horizon comparison between the two models is highly competitive rather than dominated by one side.

Autoformer is a stable third-place baseline across all 12 horizons. However, its overall average RMSE of 38.71 is far above the two leading models, whose overall average RMSE values are 16.04 for S_Mamba and 16.15 for iTransformer. The plain attention baselines are substantially weaker on GLD under the locked shared setting, with overall average RMSE above 69 for Transformer and above 76 for Informer and Reformer.

The MAE pattern is similar. S_Mamba and iTransformer remain nearly tied overall, and S_Mamba holds a slightly lower average MAE, 11.61 versus 11.65. These baseline results identify S_Mamba and iTransformer as the two reference models that NS-Mamba must improve upon.

6.1 NS-Mamba Diagnostic Ablation

The first NS-Mamba diagnostic sweep is complete for horizons 30, 120, 180, and 360 days. This is a partial ablation rather than the final 12-horizon result table. The completed diagnostic stage contains 16 finished runs, corresponding to four NS-Mamba variants evaluated on four horizons under the same locked Setting A used by the GLD baselines.

The strongest diagnostic variant is NS-Mamba-D, which uses series decomposition with the S_Mamba normalization pathway and does not use adaptive normalization or shift-aware fusion. Against the better of S_Mamba and iTransformer at each diagnostic horizon, NS-Mamba-D reduces RMSE by 13.13% at 30 days, 8.25% at 120 days, 9.33% at 180 days, and 4.33% at 360 days. Its long-horizon average RMSE is 26.74, compared with 28.90 for S_Mamba and 28.68 for iTransformer. This corresponds to a 7.49% reduction versus S_Mamba and a 6.79% reduction versus iTransformer. The diagnostic result is also informative because the more complex variants do not improve the

Table 5: Direct RMSE and MAE comparison of NS-Mamba-D against all completed GLD models on the overlapping horizons 30, 120, 180, and 360 days. Long Avg. is computed over 120, 180, and 360 days.

Track	Model	30	120	180	360	Long Avg. RMSE	Long Avg. MAE
Opt	PatchTST	9.65	15.32	21.23	33.33	23.29	16.94
Opt	NLinear	9.45	17.53	24.09	34.31	25.31	18.08
Opt	TimesNet	9.40	16.33	24.81	36.63	25.92	18.90
NS-Diag	**NS-Mamba-D**	9.66	18.25	24.22	37.74	26.74	19.54
Shared	iTransformer	11.45	19.89	26.71	39.45	28.69	20.68
Shared	S_Mamba	11.12	20.31	26.91	39.48	28.90	20.84
Opt	Non-stationary Transformer	9.63	20.40	26.76	39.85	29.00	19.99
NS-Diag	NS-Mamba-AD	10.46	20.50	27.49	42.39	30.13	22.34
NS-Diag	NS-Mamba-ADG	10.64	20.19	27.60	45.50	31.10	23.16
NS-Diag	NS-Mamba-A	11.16	21.16	28.40	43.93	31.16	22.97
Opt	DLinear	12.93	27.53	35.23	51.14	37.97	27.13
Shared	Autoformer	37.96	41.02	42.99	44.35	42.79	29.01
Shared	Transformer	66.85	66.08	69.12	81.22	72.14	59.64
Shared	Informer	77.45	72.34	68.18	78.79	73.11	56.60
Shared	Reformer	73.82	81.33	85.29	83.92	83.51	67.99

metric. Adaptive normalization alone is weaker than S_Mamba on the diagnostic long horizons, and adding adaptive normalization to the decomposition branch weakens the decomposition-only result. The shift-aware gate also does not help in this first diagnostic sweep. Therefore, the next experiment should promote NS-Mamba-D to the full 12-horizon GLD evaluation before adding further complexity.

6.2 NS-Mamba-D vs All Completed Models

Table 5 compares NS-Mamba-D against every model that currently has completed GLD results on the overlapping horizons 30, 120, 180, and 360 days. This includes the shared locked-setting baselines, the other NS-Mamba diagnostic variants, and the tuned imported baselines.

NS-Mamba-D ranks 5th at 30 days, 4th at 120 days, 3rd at 180 days, and 4th at 360 days among all completed models. On the long-horizon average over 120, 180, and 360 days, NS-Mamba-D ranks 4th overall with RMSE 26.74 and MAE 19.54. It beats every shared locked-setting baseline and every other NS-Mamba diagnostic variant, but it trails the tuned imported PatchTST, NLinear, and TimesNet baselines. This comparison predates the later Mamba-family hyperparameter optimization; after that search, optimized MS-Mamba-MD becomes the strongest Mamba-family result.

6.3 Source-Optimized Imported Baselines

We also ran a separate source-style optimized sweep for imported comparison models using the hyperparameter ranges from the moved `nonstationary_transformer/scripts/GLD.sh` workflow. This sweep is intentionally different from the locked shared-setting baseline table because it uses model-specific hyperparameters and a sequence-length sweep. At the current stage, the optimized sweep is complete for the five imported models Non-stationary Transformer, TimesNet, PatchTST, DLinear, and NLinear.

The completed optimized imported sweep changes the comparison picture substantially. NLinear is the strongest imported model on the short horizons from 1 to 14 days. TimesNet is strongest

Table 6: Completed source-optimized GLD coverage for the imported models. Autoformer and Transformer source-style sweeps are still pending.

Model	Sweep rows	Best rows	Max horizon
PatchTST	108	12	360
NLinear	108	12	360
TimesNet	108	12	360
Non-stationary Transformer	108	12	360
DLinear	108	12	360

Table 7: Best completed source-optimized imported results by horizon, measured with RMSE and paired with MAE.

Horizon	Winner	Best seq_len	RMSE	MAE
1	NLinear	320	2.56	1.70
3	NLinear	320	3.53	2.36
5	NLinear	384	4.20	2.85
7	NLinear	192	4.83	3.30
14	NLinear	192	6.54	4.55
30	TimesNet	256	9.40	6.89
60	PatchTST	320	13.34	9.76
90	TimesNet	320	13.29	10.23
120	PatchTST	256	15.32	11.61
180	PatchTST	320	21.23	15.86
270	PatchTST	128	28.13	20.23
360	PatchTST	128	33.33	23.36

at 30 and 90 days. PatchTST is strongest from 60 days onward except for 90 days, and it is the best imported model overall under the completed optimized sweep.

The best lookback pattern also depends strongly on the model. NLinear prefers very large windows from 192 to 384. PatchTST prefers 16 on short horizons, then 256 to 320 at middle horizons, and 128 at 270 and 360 days. TimesNet prefers 16 on short horizons and much larger windows at middle and long horizons. Non-stationary Transformer still prefers mostly 16, which appears too short for strong long-horizon GLD performance.

Compared with the best locked-setting baseline at each completed horizon, the optimized imported baselines are substantially stronger except for DLinear. For example, optimized NLinear reduces RMSE by 42.41% at 1 day, optimized TimesNet reduces RMSE by 20.36% at 90 days, optimized PatchTST reduces RMSE by 23.02% at 120 days, and optimized PatchTST reduces RMSE by 15.51% at 360 days. DLinear is weaker than the best locked-setting baseline at every horizon, while Non-stationary Transformer is competitive at short horizons but no longer competitive with the strongest imported models once the full optimized sweep is complete.

6.4 Mamba Hyperparameter Optimization

The Mamba-family hyperparameter optimization completed 266 price-scale runs and selected 25 best rows by RMSE. The search covers S_Mamba, NS-Mamba-D, MS-Mamba variants, and PMS-Mamba-PD over diagnostic horizons. Because the promoted Mamba search has not yet run all 12 horizons, the direct long-horizon comparison below uses the completed diagnostic long horizons 120, 180, and 360 days.

Table 8: Long-horizon comparison after Mamba hyperparameter optimization. Values are averaged over 120, 180, and 360 days.

Track	Model	Avg. RMSE	Avg. MAE
Opt	PatchTST	23.29	16.94
Mamba-HPO	MS-Mamba-MD	24.45	17.97
Mamba-HPO	NS-Mamba-D	24.83	18.25
Mamba-HPO	MS-Mamba-MDG	25.00	18.28
Opt	NLinear	25.31	18.08
Mamba-HPO	S_Mamba	25.40	18.60
Mamba-HPO	MS-Mamba-MDS	25.78	18.40
Opt	TimesNet	25.92	18.90
Mamba-HPO	PMS-Mamba-PD	25.93	18.62

Table 9: Best Mamba-HPO configuration by completed diagnostic horizon.

Model	Horizon	Seq. Len	RMSE	MAE	Config	Moving Avg.	Scales
MS-Mamba-MDG	30	256	9.02	6.61	MSG25S124	25	1-2-4
MS-Mamba-MD	120	320	16.32	12.77	MSD49S124	49	1-2-4
MS-Mamba-MD	180	360	21.47	16.20	MSD49S124	49	1-2-4
MS-Mamba-MD	360	360	35.55	24.95	MSD49S124	49	1-2-4

After tuning, MS-Mamba-MD is the strongest Mamba-family model, with average RMSE 24.45 and average MAE 17.97 over the completed long diagnostic horizons. This changes the cross-model ranking: the best optimized Mamba model is now stronger than optimized NLinear and TimesNet on the 120/180/360-day average. However, optimized PatchTST remains the strongest model in this comparison, with average RMSE 23.29; the remaining relative RMSE gap for the best Mamba model is 4.96%. The residual gap to PatchTST is concentrated at 120 days (16.32 vs. 15.32), 180 days (21.47 vs. 21.23), 360 days (35.55 vs. 33.33).

6.5 Completed Result Overview

Figure 2 replaces the oversized full-results table with a compact visual summary of the completed GLD comparisons. Panel A shows the main cross-model frontier, and Panel B shows the completed Mamba-family comparisons. All curves are plotted on the original GLD price scale using RMSE. The local data snapshots used by this figure are bundled in the submission package under `figures/`.

There are several plausible reasons why Autoformer, Transformer, Informer, and Reformer are much weaker than S_Mamba and iTransformer on this GLD task. First, the two strongest models use an inverted embedding scheme that maps the full historical window into variable tokens before forecasting, while the four weaker baselines use the standard time-step sequence embedding with encoder-decoder style generation. For a single-target task with a long lookback window, the inverted formulation appears to be better aligned with direct horizon projection than the standard decoder-based setup. Second, S_Mamba and iTransformer both apply per-window mean and variance normalization with final de-normalization, which is likely important for GLD because the price level and volatility change substantially across time. Transformer, Informer, and Reformer in this repository do not apply the same reversible normalization path, and Autoformer applies decomposition but not the same explicit nonstationary normalization. Third, the baseline comparison

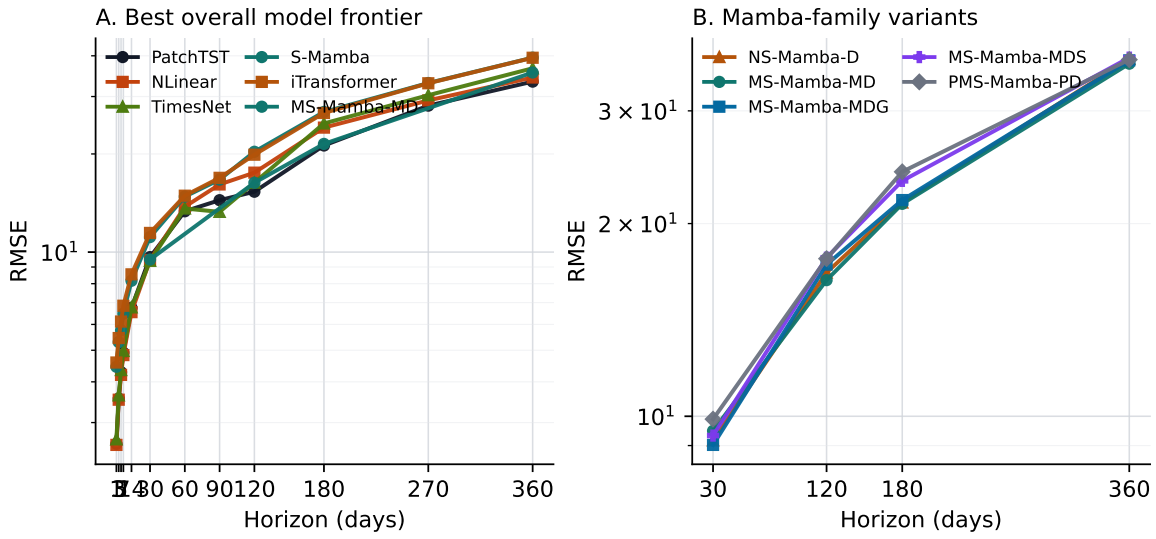


Figure 2: Compact GLD RMSE comparison for publication. Panel A shows the main cross-model frontier across all completed horizons, including the strongest shared baselines, tuned imported baselines, and the promoted Mamba-HPO result. Panel B compares the completed Mamba-family variants on the diagnostic horizons 30, 120, 180, and 360 days. This figure preserves the ranking structure without the space cost of the full result table.

deliberately uses one shared setting selected around S_Mamba, so the result should be interpreted as a controlled comparison under one configuration rather than as a per-model hyperparameter optimum. Even with that caveat, the size and consistency of the gap suggest that architecture-data mismatch, not only tuning, is a major factor. The completed source-optimized imported sweep reinforces this point from the opposite direction: once model-specific hyperparameters are allowed, PatchTST, NLinear, and TimesNet all become much stronger than their locked-setting counterparts, which means GLD is unusually sensitive to lookback and model-scale choices. The same conclusion holds for the Mamba family: after Mamba-specific hyperparameter optimization, the best MS-Mamba variant moves ahead of optimized NLinear and TimesNet on the completed long-horizon diagnostic average, although it still trails optimized PatchTST.

7 Ablation Study

The ablation study isolates the contribution of each NS-Mamba component. The current diagnostic comparison is:

Model	Adaptive Norm	Decomposition	Shift Gate
S_Mamba	No	No	No
NS-Mamba-A	Yes	No	No
NS-Mamba-D	No	Yes	No
NS-Mamba-AD	Yes	Yes	No
NS-Mamba-ADG	Yes	Yes	Yes

The first diagnostic result supports the decomposition hypothesis but does not support the adaptive-normalization or shift-gating additions in their current form. NS-Mamba-D improves over

both S_Mamba and iTransformer on the diagnostic long-horizon average. After Mamba-specific hyperparameter optimization, however, the strongest Mamba-family result is MS-Mamba-MD rather than NS-Mamba-D. This suggests that decomposition remains useful, but multi-scale temporal resolution and lookback selection are at least as important as the original nonstationary gating idea for this GLD task. The full 12-horizon Mamba-HPO run is still required before making a final claim about the optimized Mamba family.

8 Discussion and Limitations

This manuscript is currently a draft after the baseline stage, the first NS-Mamba diagnostic stage, and a Mamba-family hyperparameter optimization stage. The main limitation is no longer baseline coverage; that part is complete. The main remaining limitation is that the best optimized Mamba variant, MS-Mamba-MD, has not yet been evaluated across all 12 GLD horizons under the HPO-selected settings. The source-optimized imported baseline sweep is complete for PatchTST, NLinear, TimesNet, Non-stationary Transformer, and DLinear, but the optional source-style Autoformer and Transformer sweeps are still pending.

The second limitation is dataset scope. GLD provides a useful nonstationary financial testbed, but one dataset is not enough to establish general forecasting superiority. After GLD experiments are complete, the method should also be tested on standard long-horizon forecasting datasets such as Exchange, Weather, ETT, Electricity, and Traffic.

The third limitation is split sensitivity. Financial time series evaluation can change substantially with different historical periods. The final manuscript should report the exact date range, train-validation-test boundaries, random seeds, and whether validation or test windows use lookback context from earlier splits.

Finally, the architecture may add complexity. The first diagnostic ablation suggests that not every nonstationary module is useful in this setting. Adaptive normalization and shift-aware fusion underperform the simpler decomposition-only branch, so future changes should be justified by improved metrics rather than by architectural novelty alone. The completed baseline table also raises the bar for improvement: the optimized Mamba model now needs to preserve its long-horizon advantage while remaining competitive at short and middle horizons. The completed optimized imported sweep raises the bar further, because NLinear is strongest at the short horizons and PatchTST is strongest overall under the imported-model tuning regime. The baseline analysis also suggests where the improvement must come from. If NS-Mamba only adds complexity without preserving the inverted-token and reversible-normalization advantages already present in the two strongest shared-setting baselines, it is unlikely to close the gap to the stronger tuned imported baselines for the right reason. The proposed model therefore needs to build on the strengths of S_Mamba while explicitly targeting the long-horizon regime where iTransformer is strongest under the shared setting and PatchTST remains strongest under the tuned imported setting.

9 Conclusion

This manuscript presents a structured path for nonstationarity-aware Mamba forecasting. The completed GLD baseline study shows that S_Mamba is the strongest short- and middle-horizon baseline, while iTransformer is the strongest long-horizon RMSE baseline under the shared locked setting. The completed source-optimized imported sweep further shows that NLinear is strongest at short horizons and PatchTST is strongest overall among the tuned imported baselines. The proposed NS-Mamba and MS-Mamba architecture families combine decomposition, multi-scale temporal mod-

eling, and Mamba-based sequence processing around an S_Mamba backbone. The first diagnostic ablation identifies the decomposition-only variant, NS-Mamba-D, as a strong candidate, reducing diagnostic long-horizon RMSE by 7.48% versus S_Mamba and 6.80% versus iTransformer. After Mamba-specific hyperparameter optimization, MS-Mamba-MD becomes the strongest Mamba-family model, reaching 24.45 average RMSE over the 120/180/360-day diagnostic horizons. This optimized Mamba result is stronger than optimized NLinear and TimesNet on the same long-horizon average, but optimized PatchTST remains best with average RMSE 23.29. The next step is to evaluate the HPO-selected MS-Mamba-MD configuration across the full 12-horizon GLD suite. The manuscript will become submission-ready after the proposed-model results, ablations, and broader benchmark evaluation are completed with traceable quantitative evidence.

References

[1] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023.

[2] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is Mamba Effective for Time Series Forecasting? arXiv:2403.11144, 2024.

[3] Aobo Liang, Xingguo Jiang, Yan Sun, Xiaohou Shi, and Ke Li. Bi-Mamba+: Bidirectional Mamba for Time Series Forecasting. arXiv:2404.15772, 2024.

[4] Xiuding Cai, Yaoyao Zhu, Xueyao Wang, and Yu Yao. MambaTS: Improved Selective State Space Models for Long-term Time Series Forecasting. arXiv:2405.16440, 2024.

[5] Md Atik Ahamed and Qiang Cheng. TimeMachine: A Time Series is Worth 4 Mambas for Long-term Forecasting. European Conference on Artificial Intelligence, 2024.

[6] Haoyu Ma, Yushu Chen, Wenlai Zhao, Jinzhe Yang, Yingsheng Ji, Xinghua Xu, Xiaozhu Liu, Hao Jing, Shengzhuo Liu, and Guangwen Yang. A Mamba Foundation Model for Time Series Forecasting. arXiv:2411.02941, 2024.

[7] Tae-Hyun Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. International Conference on Learning Representations, 2022.

[8] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary Transformers: Exploring the Stationarity in Time Series Forecasting. Advances in Neural Information Processing Systems, 2022.

[9] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. International Conference on Learning Representations, 2024.

[10] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI Conference on Artificial Intelligence, 2021.

[11] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. Advances in Neural Information Processing Systems, 2021.

[12] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The Efficient Transformer. International Conference on Learning Representations, 2020.